AutoPatch

Self-Improving Multi-Agent Code Repair System

Multi-Agent | RL-Powered | Self-Improving

Author	Version	Status	Date
Thowfiq	1.0	Draft	April 2026

1. Product Overview

AutoPatch is a multi-agent AI system that autonomously detects, patches, and validates Python bugs using a custom reinforcement learning environment (CodeReview-Env). Five specialized agents collaborate in a LangGraph pipeline, and the system improves its patching strategy across episodes through reward-driven prompt evolution.

1.1 Problem Statement

Every software team runs CI/CD pipelines that block merges when tests fail. Engineers spend hours triaging test failures and writing fixes — work that is repetitive, well-specified, and ripe for automation. Existing benchmarks like SWE-bench measure performance but are not designed for RL training. AutoPatch fills this gap by combining a purpose-built RL environment with a collaborative agent system that gets smarter over time.

1.2 Core Value Propositions

- Autonomous bug detection and patching without human intervention
- Dense reward signal enables continuous self-improvement across episodes
- Multi-agent collaboration with specialised roles (Planner, Coder, Critic, Memory, Evolver)
- Production-ready deployment with FastAPI backend and React dashboard
- GitHub integration for real open-source issue resolution (v2)

1.3 System Architecture

Layer	Components
Frontend	React + Recharts dashboard — reward curves, live agent logs, episode viewer
API Layer	FastAPI server — /run, /logs (SSE), /rewards, /status endpoints
Orchestrator	LangGraph StateGraph — manages episode loop, routes between agents
Agent Layer	Planner → Coder → Critic → Memory → Evolver (5 specialised agents)
Environment	Your CodeReview-Env — /reset, /step, /state (never modified)
Memory Store	Redis (JSON fallback) — episode trajectories, fix patterns, logs
LLM Provider	Groq API (llama-3.3-70b-versatile) — powers all agent reasoning

2. Environment Contract — CodeReview-Env

AutoPatch treats CodeReview-Env as an immutable external service. All agents communicate with it exclusively via HTTP. The environment must never be modified.

2.1 API Contract

Endpoint	Method	Description
`POST /reset`	POST	Starts new episode. Returns CodeObservation with task_id, context, available_files
`POST /step`	POST	Executes one action. Body: AgentAction JSON. Returns observation, reward, done
`GET /state`	GET	Returns current ReviewState — episode_id, step_count, total_reward, done
`GET /health`	GET	Returns {status: healthy}. Must respond before any episode starts
`GET /metadata`	GET	Returns env name, description, version
`GET /schema`	GET	Returns JSON schemas for action, observation, state models
`POST /mcp`	POST	JSON-RPC 2.0 — tools/list method supported

2.2 Reward Structure

- execute_command → always 0.0 (exploration is free)
- patch_file (no tests gained) → 0.0
- patch_file (partial fix) → (tests_passing / total_tests) × 0.9
- submit_review (all pass) → 1.0 (terminal)
- submit_review (partial) → tests_passing / total_tests
- Max steps exceeded (10) → 0.0, episode terminates

2.3 Action Space

```
{"action_type": "execute_command", "command": "pytest tests/ -v"}
{"action_type": "patch_file", "target_file": "auth/crypto.py", "new_content": "..."}
{"action_type": "submit_review", "summary": "Fixed SQL injection and MD5."}
```

3. File-by-File Product Requirements

Every file below is a first-class product requirement. Each has a purpose, detailed subtasks, constraints, and acceptance criteria. Build them in the order listed.

3.1 autopatch/agents/planner.py

`autopatch/agents/planner.py`

Description: The Planner agent reads pytest failure output and PR context to produce a structured fix plan.

Purpose: First node in the LangGraph pipeline. Determines what to fix and in what order before any code is written.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`plan(obs)`	Accept a CodeObservation dict. Extract context, action_result (pytest output), available_files.	Function accepts dict, does not crash on missing keys (use .get()).	P0 — Critical
`LLM call`	Use ChatGroq with model='llama-3.3-70b-versatile', temperature=0.0 for determinism.	Same input always produces same output. Model must be configurable via env var.	P0 — Critical
`System prompt`	Instruct LLM to return ONLY valid JSON array. No markdown, no preamble.	Raw response must parse with json.loads() without any stripping except ```json fence removal.	P0 — Critical
`Output schema`	Return list of dicts: [{file, bug_description, fix_strategy, bug_type}]. bug_type must be one of: sql_injection, weak_crypto, logic_error, hardcoded_secret, async_error.	Output validates against schema. Unknown bug_type defaults to 'logic_error'.	P0 — Critical
`Error handling`	Wrap LLM call in try/except. On JSON parse failure, return empty list — never raise.	Orchestrator never crashes due to planner failure. Logs error message.	P1 — High
`Order priority`	Bugs that block more tests should appear first in the output array.	If task_1 has SQL + crypto bugs, SQL fix (unblocking 1 test) comes before crypto (1 test).	P2 — Medium
`File filtering`	Only include files that exist in available_files list from observation.	Planner never outputs a fix for a file not present in the sandbox.	P1 — High

3.2 autopatch/agents/coder.py

`autopatch/agents/coder.py`

Description: The Coder agent takes a single fix task and generates a complete corrected file using an LLM.

Purpose: Core patch generation. Produces the new_content that gets submitted to patch_file. Supports dynamic prompt evolution from the Evolver agent.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority

`code()`	Accept: file_content (str), task (dict from planner), memory_hint (str, optional). Return: {target_file, new_content, explanation}.	Function signature matches exactly. memory_hint is optional kwarg defaulting to empty string.	P0 — Critical
`Mutable prompt`	System prompt stored as module-level _current_system variable. Expose set_system(), get_system(), reset_system() functions.	Evolver can call set_system() and next code() call uses the new prompt without restart.	P0 — Critical
`Return entire file`	LLM must return the COMPLETE corrected file, not just the changed lines.	new_content when written to disk passes py_compile without errors.	P0 — Critical
`Bug-type rules`	System prompt must include explicit rules per bug type: SQL→parameterized tuple, Crypto→pbkdf2_hmac never md5, Secrets→os.getenv(), Async→async def + await.	Critic approves patches that follow these rules on first attempt (>80% of the time).	P0 — Critical
`Memory hint injection`	If memory_hint is non-empty, append to user message: 'Similar past fix that worked: {hint}' (truncated to 800 chars).	When memory hint is present, coder reuses the same pattern rather than inventing a new one.	P1 — High
`JSON output only`	Strip ```json fences from LLM response before parsing. Raise ValueError if result is not a dict with target_file and new_content keys.	Critic never receives a malformed patch object.	P0 — Critical
`Retry support`	Critic feedback is passed back as additional user message in a second LLM call if approved=False.	On retry, coder incorporates the critic's specific feedback into the new patch.	P1 — High

3.3 autopatch/agents/critic.py

`autopatch/agents/critic.py`

Description: The Critic agent reviews a proposed patch before it is applied to the environment.

Purpose: Quality gate between the Coder and the environment. Prevents wasted patch_file calls and 0.0 rewards from syntactically correct but semantically wrong patches.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`critique()`	Accept: original (str — current file content), patch (dict — coder output), bug_type (str). Return: {approved: bool, feedback: str, confidence: float}.	Function signature matches. Never raises — returns {approved: False, feedback: 'error', confidence: 0} on failure.	P0 — Critical
`SQL check`	Reject if new_content contains f-string SQL (f"SELECT... {username}"). Approve only if returns a tuple (query, (param,)).	Patches with f-string SQL are rejected 100% of the time.	P0 — Critical
`Crypto check`	Reject if new_content contains 'md5' or 'sha1'. Approve only if uses pbkdf2_hmac, bcrypt, or argon2.	Patches using MD5 are rejected 100% of the time.	P0 — Critical
`Secret check`	Reject if new_content contains any string matching /sk_[a-z0-9]+/ or /pk_[a-z0-9]+/ or /key_[a-z0-9]+/ as a literal value.	Hardcoded API keys in new_content are always rejected.	P0 — Critical
`Async check`	Reject if bug_type=async_error and new_content does not contain both 'async def process_payment' AND 'await'.	A syntactically valid 'def process_payment' with await but no async is rejected.	P0 — Critical
`Max retries`	Orchestrator must limit critic rejections to 2 retries per task. After 2 rejections, proceed with the best available patch.	Episodes never stall in Coder-Critic loop. Max 3 total coder calls per task (1 + 2 retries).	P0 — Critical

`Confidence score`	Return confidence 0.0-1.0. Confidence < 0.5 with approved=True should trigger a warning log.	Low-confidence approvals are logged with [CRITIC WARNING] tag.	**P2 — Medium**

3.4 autopatch/agents/memory.py

`autopatch/agents/memory.py`

Description: Persistent storage layer for fix patterns, episode trajectories, and run logs.

Purpose: Enables the Evolver to analyse historical performance and the Coder to reuse successful past fixes. Must work with or without Redis.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`Redis detection`	On import, attempt redis.from_url(REDIS_URL).ping(). Set USE_REDIS=True on success, False on any exception.	System boots without Redis installed. Falls back to JSON file silently.	**P0 — Critical**
`store_pattern()`	Accept: bug_type (str), patch_content (str), reward (float). Store with timestamp. Key format: pattern:{bug_type}:{md5_hash[:8]}.	Patterns are stored without overwriting different patches of the same bug type.	**P0 — Critical**
`retrieve_pattern()`	Return best past patch for bug_type where reward >= 0.9. Return empty string if none found. Truncate to 800 chars.	Returns empty string (not None, not exception) when no match found.	**P0 — Critical**
`Redis list cap`	Use LPUSH + LTRIM to keep max 50 patterns per bug_type in Redis.	Redis memory does not grow unboundedly. Old patterns are evicted automatically.	**P1 — High**
`store_trajectory()`	Accept: run_id, episode (int), rewards (list[float]), score (float). Store full trajectory for Evolver analysis.	Evolver can retrieve last N trajectories for any run_id.	**P0 — Critical**
`store_log() / get_logs()`	Append single log line to run_id log. get_logs returns ordered list. Used by SSE endpoint.	Dashboard receives logs in the exact order they were stored.	**P0 — Critical**
`JSON fallback path`	Fallback file path: autopatch/episodes.json. Create parent dirs if missing. All operations on this file use atomic write (write to tmp, rename).	Concurrent writes do not corrupt episodes.json.	**P1 — High**

3.5 autopatch/agents/evolver.py

`autopatch/agents/evolver.py`

Description: The Evolver agent analyses reward trajectories and rewrites the Coder's system prompt to improve future performance.

Purpose: The self-improvement engine. This is what makes AutoPatch unique — agents that get measurably better over time. The reward curve going up is the demo.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority

`maybe_evolve()`	Accept: run_id (str), episode_number (int). Trigger only when episode_number % 5 == 0. No-op otherwise.	Evolver runs exactly on episodes 5, 10, 15, 20... Never on other episodes.	**P0 — Critical**
`Min trajectories`	Require at least 3 stored trajectories before evolving. Return silently if fewer exist.	Evolver never runs with insufficient data to produce a meaningful prompt.	**P0 — Critical**
`Trajectory summary`	For each trajectory, compute: steps_to_first_reward (index of first reward > 0), total steps, final score. Pass this summary to LLM.	LLM prompt includes quantitative performance data, not just raw reward arrays.	**P1 — High**
`Prompt validation`	New prompt must be >200 characters before applying. Discard LLM outputs that are too short.	Corrupted or empty LLM responses never replace the working coder prompt.	**P0 — Critical**
`Log evolution`	[EVOLVER] tag in log with episode number and number of trajectories analysed.	Every prompt update is traceable in the dashboard log.	**P1 — High**
`Temperature`	Use temperature=0.3 (not 0.0) for evolution LLM calls — allows creative prompt variation.	Evolved prompts are meaningfully different from the base prompt, not identical.	**P2 — Medium**
`Preserve rules`	System prompt given to LLM must instruct it to keep all existing bug-type rules and only ADD guidance.	Evolved prompt never loses SQL/crypto/async rules that were in the base prompt.	**P0 — Critical**

3.6 autopatch/orchestrator.py

autopatch/orchestrator.py

Description: LangGraph StateGraph that defines the full episode loop: reset → plan → read → code → critique → patch → submit.

Purpose: The brain of the system. Routes between agents, handles retries, manages state across all nodes, and calls memory/evolver after each episode.

State Schema — EpisodeState

Field	Type	Description
run_id	str	Unique identifier for this run. Passed to memory for all storage operations.
episode	int	Episode number within the run. Used by Evolver to decide when to evolve.
observation	dict	Latest CodeObservation from the environment. Updated after every /step call.
plan	list	List of fix tasks from the Planner. Persists across all nodes in the episode.
current_task_index	int	Which task in the plan is currently being worked on. Increments after each patch.
current_patch	dict	The patch generated by the Coder. Includes _review key after Critic runs.
original_file	str	Content of the file read before patching. Used by Critic for comparison.
rewards	list	All rewards received this episode. Final score = max(rewards).
done	bool	True when environment returns done=True or plan is exhausted.
retry_count	int	Number of Critic rejections for current task. Resets to 0 after each new task.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
node_reset	Call POST /reset. Then immediately call POST /step with execute_command 'pytest tests/ -v' to get initial failure output. Return as observation.	Observation passed to Planner always contains pytest output, not the bare /reset response.	P0 — Critical
node_plan	Call planner.plan(observation). On empty result or exception, set done=True and skip to submit.	Episode never hangs when planner returns empty plan. Proceeds to submit with score 0.	P0 — Critical
node_read_file	Call execute_command 'cat {task.file}'. Store result as original_file. If task index >= len(plan), set done=True.	Original file content is always populated before Coder runs.	P0 — Critical
node_code	Call coder.code() with file content, task, and memory hint. On exception, set done=True.	Coder exceptions do not crash the episode. They terminate it gracefully with current score.	P0 — Critical
node_critic	Call critic.critique(). Attach review dict to current_patch under '_review' key.	Review is always present in current_patch before routing decision.	P0 — Critical
node_patch	Call POST /step with patch_file action. Store reward. Increment current_task_index. Call memory.store_pattern() if reward > 0.	Every successful patch (reward > 0) is stored in memory immediately.	P0 — Critical
node_submit	Call POST /step with submit_review. Compute final score = max(rewards). Call memory.store_trajectory(). Call evolver.maybe_evolve().	Evolver is always called after submit, even on failed episodes.	P0 — Critical

`Routing: after critic`	approved=True OR retry_count >= 2 → go to patch. approved=False AND retry_count < 2 → increment retry_count, go back to code.	Coder-Critic loop terminates after max 2 retries. Never infinite.	P0 — Critical
`Routing: after patch`	done=True → END. current_task_index >= len(plan) → submit. Otherwise → read_file for next task.	All routing conditions are mutually exclusive and exhaustive.	P0 — Critical
`httpx timeouts`	All HTTP calls to ENV_URL must use timeout=30 seconds.	Network hangs do not stall episodes indefinitely.	P1 — High

3.7 autopatch/server.py

`autopatch/server.py`

Description: FastAPI server that exposes the AutoPatch system as an HTTP API for the dashboard and external consumers.

Purpose: Bridges the orchestrator and the React dashboard. Manages concurrent runs, streams logs via SSE, and exposes reward data for charting.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`POST /run`	Accept episodes (int, default=10) as query param. Generate run_id (UUID first 8 chars). Start background asyncio task. Return {run_id, episodes} immediately.	Response arrives in < 100ms regardless of how long episodes take.	P0 — Critical
`Background task`	Use asyncio.create_task(). Run episodes sequentially with asyncio.to_thread(orchestrator.run_episode). Update _active_runs dict after each episode.	UI can poll /status and see episode progress in real time.	P0 — Critical
`GET /status/{run_id}`	Return: status (running|done|not_found), episode (current), average_score, scores (list of all episode scores so far).	Dashboard can render a reward chart from /status alone, without needing /rewards.	P0 — Critical
`GET /rewards/{run_id}`	Call memory.get_trajectories(). Return full trajectory data including per-step rewards for each episode.	Detailed step-level reward chart is possible from this endpoint.	P1 — High
`GET /logs/{run_id} SSE`	Stream logs as Server-Sent Events. Poll memory.get_logs() every 0.5s. Send 'data: [STREAM_END]' when run is done and all logs are sent.	Dashboard EventSource receives every log line in order with no duplicates.	P0 — Critical
`CORS`	Allow all origins (allow_origins=['*']). Dashboard on localhost:5173 must be able to call API on localhost:8000.	Browser does not block any dashboard API call with CORS error.	P0 — Critical
`GET /health`	Return {status: healthy}. No dependencies — must respond even if Redis is down.	Health check never fails due to downstream service issues.	P1 — High
`Error handling`	All endpoints wrapped in try/except. Return {error: message} with 500 status on unhandled exceptions.	Server never returns an unhandled 500 with traceback to the client.	P1 — High

3.8 autopatch/run.py

`autopatch/run.py`

Description: CLI entry point for running AutoPatch from the terminal without the dashboard.

Purpose: Used for quick testing, CI integration, and reproducing results. Output format matches inference.py from CodeReview-Env for consistency.

Subtasks & Constraints

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`CLI args`	Accept --episodes (int, default=3) and --env-url (str, default=http://localhost:7860) via argparse.	python -m autopatch.run --episodes 5 --env-url http://localhost:7860 runs without error.	P0 — Critical
`Output format`	Print [AUTOPATCH] on start, [SUMMARY] after each episode with score, [AUTOPATCH] on completion. Matches [START]/[STEP]/[END] format from existing inference.py.	Log output is parseable by the same tools used to parse inference.py output.	P0 — Critical
`run_id generation`	Generate UUID run_id at start. Print it on first line so logs can be retrieved via API later.	Run results are findable in memory store by run_id printed to terminal.	P1 — High
`env var override`	Set os.environ['ENV_URL'] = args.env_url before importing orchestrator (import is deferred).	ENV_URL used by all agents reflects the CLI argument, not the .env default.	P0 — Critical
`Exit code`	Exit 0 on completion. Exit 1 if ENV_URL health check fails before starting.	CI systems can detect failures via exit code.	P2 — Medium

3.9 autopatch/dashboard/src/App.jsx

`autopatch/dashboard/src/App.jsx`

Description: Single-page React dashboard for monitoring AutoPatch runs in real time.

Purpose: Primary user interface. Shows reward curves, live agent logs, run statistics, and episode controls.

Components & Subtasks

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`RunButton`	POST /run with episodes count. Disable while running. Show current episode progress in button label.	Button is disabled immediately on click. Re-enables only when status.status === 'done'.	P0 — Critical
`RewardChart`	LineChart from recharts. X-axis: episode number. Y-axis: score 0.0-1.0. Updates live via /status polling every 1 second.	Chart updates in real time as episodes complete. Does not require page refresh.	P0 — Critical
`StatCards`	Three stat cards: Episodes Run, Average Score, Perfect (1.0) count. Computed from scores array.	Cards update every time /status poll returns new data.	P1 — High
`AgentLog`	Scrolling log panel fed by EventSource on /logs/{run_id}. Colour-coded by tag: [END]=green, [CRITIC]=yellow, [EVOLVER]=purple, [MEMORY]=blue, ERROR=red.	Log auto-scrolls to bottom on new entries. Colours match tag exactly.	P0 — Critical
`SSE handling`	Open EventSource on /logs/{run_id} when run_id is set. Close on [STREAM_END] or component unmount.	EventSource is never left open after run completes (no memory leak).	P0 — Critical
`Status polling`	useEffect with setInterval(1000) on runId. Clear interval when status.status === 'done'.	Polling stops automatically. Browser does not make unnecessary requests after run ends.	P0 — Critical
`Styling`	Tailwind CSS. Dark theme (bg-gray-950). Monospace font for logs. Green accent (green-400) for scores and primary actions.	Dashboard is usable in dark mode. No white flash on load.	P1 — High
`API_URL config`	API base URL defined as top-level const. Default: http://localhost:8000.	Changing API URL requires editing only one line.	P2 — Medium

4. GitHub Integration — v2 Features

These two files extend AutoPatch to work with real GitHub issues and open pull requests automatically. Build these after the core system is working end-to-end.

4.1 autopatch/github/ingester.py

`autopatch/github/ingester.py`

Description: Fetches a real GitHub issue and converts it into a CodeReview-Env compatible task.

Purpose: Enables AutoPatch to work on real open-source bugs, not just the 3 built-in tasks.

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`URL parsing`	Parse https://github.com/owner/repo/issues/N using regex. Raise ValueError with helpful message on invalid URL.	Invalid URL raises ValueError('Invalid GitHub issue URL — expected https://github.com/owner/repo/issues/N').	P0 — Critical
`Auth`	Read GITHUB_TOKEN from env var. Pass to Github() constructor. If missing, raise EnvironmentError.	No GitHub API calls are made without authentication.	P0 — Critical
`File extraction`	Extract file mentions from issue body using regex: any `filename.py` in backticks.	Returns at minimum an empty list, never None or exception on bodies with no file mentions.	P1 — High
`Output format`	Return dict with: issue_number, title, context (full body), available_files (list), repo (owner/repo), url.	Output dict can be passed directly as observation to planner.plan().	P0 — Critical
`Rate limit`	If Github raises RateLimitExceededException, catch and raise RuntimeError with wait time.	User sees a helpful error message instead of a raw exception.	P2 — Medium

4.2 autopatch/github/pr_opener.py

`autopatch/github/pr_opener.py`

Description: Opens a pull request with AutoPatch's fixes when an episode achieves reward = 1.0.

Purpose: Closes the loop — agents not only fix bugs locally but contribute back to real repositories.

Subtask	Requirement / Constraint	Acceptance Criteria	Priority
`Branch creation`	Branch name: autopatch/fix-{issue_number}. If branch already exists (409), continue without error.	PR opener is idempotent — running twice on same issue doesn't crash.	P0 — Critical
`File commits`	For each patched file: try get_contents() then update_file(). On 404, use create_file(). Both use branch=branch_name.	Files are committed to the correct branch, not main.	P0 — Critical
`PR body`	Include: agent system description, final score, reward trajectory formatted as 0.00 → 0.45 → 0.90 → 1.00, link to AutoPatch repo.	PR body is human-readable and explains what the agents did.	P1 — High
`Return value`	Return the PR html_url string. Caller can display this in dashboard and terminal.	Dashboard shows a clickable PR link when PR is successfully opened.	P1 — High

`Guard`	Only call this function when max(rewards) == 1.0. Orchestrator must enforce this guard.	No PR is opened for failed or partial episodes.	**P0 — Critical**

5. Environment & Configuration

5.1 Required Environment Variables

Variable	Description	Default
`GROQ_API_KEY`	Groq API key — required for all LLM calls. Get free at console.groq.com.	No default — must be set
`ENV_URL`	Base URL of your CodeReview-Env instance.	http://localhost:7860
`REDIS_URL`	Redis connection string. System falls back to JSON if Redis is unreachable.	redis://localhost:6379
`GITHUB_TOKEN`	GitHub personal access token with repo scope. Required only for v2 features.	No default — optional
`MODEL_NAME`	Groq model name for all agents.	llama-3.3-70b-versatile

5.2 Startup Order

1. **Start CodeReview-Env:** `docker run -p 7860:7860 codereview-env`
2. **Verify health:** `curl http://localhost:7860/health → {status: healthy}`
3. **Start Redis (optional):** `redis-server`
4. **Start API server:** `uvicorn autopatch.server:app --port 8000 --reload`
5. **Start dashboard:** `cd autopatch/dashboard && npm run dev`
6. **Open dashboard:** `http://localhost:5173`

6. System-Level Acceptance Criteria

#	Criterion	How to verify	Priority
AC-01	Running python -m autopatch.run --episodes 1 against a live CodeReview-Env produces [START]...[END] output with score in [0.0, 1.0].	Run command, check output.	P0 — Critical
AC-02	After 3 episodes, memory.get_trajectories(run_id) returns 3 trajectory records with correct reward arrays.	Inspect episodes.json or Redis after run.	P0 — Critical
AC-03	After episode 5, coder._current_system is different from the BASE_SYSTEM constant.	Print coder.get_system() after episode 5.	P0 — Critical
AC-04	Dashboard reward chart updates in real time — new data point appears within 2 seconds of episode completion.	Watch chart while run is in progress.	P0 — Critical
AC-05	Dashboard log panel shows colour-coded lines with [PLAN], [CODE], [CRITIC], [STEP], [END] tags.	Run 1 episode, observe dashboard.	P1 — High
AC-06	Critic rejects a patch with SQL f-string and coder produces a corrected parameterized version on retry.	Manually test with a known bad patch.	P0 — Critical
AC-07	Memory hint is passed to coder on second episode with same bug type.	Add logging to coder.code() and observe.	P1 — High
AC-08	Server /logs SSE stream sends [STREAM_END] within 5 seconds of run completion.	Open /logs in browser EventSource, watch for terminal event.	P0 — Critical
AC-09	System runs 10 episodes without crashing even if individual episodes fail with score 0.	Run --episodes 10, verify [SUMMARY] line appears for every episode.	P0 — Critical
AC-10	Average score across 10 episodes is >= 0.7 (agents occasionally fix all bugs).	Check average_score in /status after 10 episodes.	P1 — High

7. Build Plan

Day	Build target	Done when
Day 1	planner.py + coder.py + orchestrator.py (no critic, no memory, no evolver)	python -m autopatch.run --episodes 1 prints [START]...[END] with a real reward
Day 2	critic.py + retry loop in orchestrator + memory.py (JSON fallback)	After 3 episodes, episodes.json exists with 3 trajectory records
Day 3	evolver.py + server.py + run.py polish	POST /run returns run_id, GET /status shows live progress, GET /logs streams
Day 4	React dashboard — chart + log + stats + run button	http://localhost:5173 shows reward curve updating in real time
Day 5 (v2)	github/ingester.py + github/pr_opener.py	Paste a GitHub issue URL, system opens a real PR on reward=1.0

8. Resume Description

AutoPatch

Built AutoPatch, a self-improving multi-agent AI system (LangGraph + Groq) that autonomously detects, patches, and validates Python bugs using a custom RL environment (CodeReview-Env). Five specialised agents — Planner, Coder, Critic, Memory, and Evolver — collaborate in a LangGraph pipeline. The Evolver agent rewrites Coder prompts based on reward trajectories, enabling measurable improvement across episodes. Achieved 1.0 reward on security (SQL injection, weak crypto), logic, and async bug tasks. Deployed as a FastAPI backend with a React dashboard showing live reward curves. GitHub integration opens automated PRs when agents achieve perfect scores.

Stack: LangGraph · Groq API · FastAPI · Redis · React · Recharts · PyGithub · Docker